Knowledge-Based Systems 21 (2008) 507–513

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

Index-BitTableFI: An improved algorithm for mining frequent itemsets

Wei Song [a,*], Bingru Yang [b], Zhangyan Xu [c]

[a] College of Information Engineering, North China University of Technology, Beijing 100144, China
[b] School of Information Engineering, University of Science and Technology, Beijing, Beijing 100083, China
[c] Department of Computer, Guanxi Normal University, Guilin 541004, China

ARTICLE INFO

Article history:
Received 24 April 2007
Accepted 18 March 2008
Available online 25 March 2008

Keywords:
Data mining
Association rule
Frequent itemset
BitTable
Index array
Subsume index

ABSTRACT

Efficient algorithms for mining frequent itemsets are crucial for mining association rules as well as for many other data mining tasks. Methods for mining frequent itemsets have been implemented using a BitTable structure. BitTableFI is such a recently proposed efficient BitTable-based algorithm, which exploits BitTable both horizontally and vertically. Although making use of efficient bit wise operations, BitTableFI still may suffer from the high cost of candidate generation and test. To address this problem, a new algorithm Index-BitTableFI is proposed. Index-BitTableFI also uses BitTable horizontally and vertically. To make use of BitTable horizontally, index array and the corresponding computing method are proposed. By computing the subsume index, those itemsets that co-occurrence with representative item can be identified quickly by using breadth-first search at one time. Then, for the resulting itemsets generated through the index array, depth-first search strategy is used to generate all other frequent itemsets. Thus, the hybrid search is implemented, and the search space is reduced greatly. The advantages of the proposed methods are as follows. On the one hand, the redundant operations on intersection of tidsets and frequency-checking can be avoided greatly; On the other hand, it is proved that frequent itemsets, including representative item and having the same supports as representative item, can be identified directly by connecting the representative item with all the combinations of items in its subsume index. Thus, the cost for processing this kind of itemsets is lowered, and the efficiency is improved. Experimental results show that the proposed algorithm is efficient especially for dense datasets.

1. Introduction

1.1. Motivation

Association rule mining, the task of finding correlations between items in a dataset, has received considerable attention, particularly since the publication of the AIS and Apriori algorithms [1,2]. Initial research was largely motivated by the analysis of market basket data, the results of which allowed companies to more fully understand purchasing behavior and, as a result, better target market audiences. For instance, if customers are buying milk, how likely are they going to also buy cereal on the same trip to the supermarket? Such information can lead to increased sales by helping retailers do selective marketing and arrange their shelf space. There are many potential application areas for association rule technology which include catalog design, store layout, customer segmentation, telecommunication alarm diagnosis, and so on.

Frequent Itemset Mining (FIM) is the most fundamental and essential problem in mining association rules. It started as a phase in the discovery of association rules, but has been generalized independent of these to many other patterns. For example, frequent sequences [3], episodes [4], and frequent subgraphs [5]. Since there are usually a large number of distinct single items in a typical transaction database, and their combinations may form a very huge number of itemsets, it is challenging to develop scalable methods for mining frequent itemsets in a large transaction database.

1.2. Related work

The Apriori algorithm [2] is the best known previous algorithm and it uses an efficient candidate generation procedure, such that only the frequent itemsets at a level are used to construct candidates at the next level. However, it requires multiple database scans, as many as the longest frequent itemset.

There have been extensive studies on the improvements or extensions of Apriori, e.g., hashing technique [6], partitioning technique [7], sampling approach [8], dynamic itemset counting [9], incremental mining [10], and integrating mining with relational database systems [11]. DCP [12] attempts to optimize itemset discovery by incorporating the dataset pruning techniques introduced in DHP and by using direct counting in the validation of candidates. CBAR [13] algorithm uses cluster table to load the databases into

* Corresponding author.
E-mail address: sgyzfr@yahoo.com.cn (W. Song).

0950-7051/$ - see front matter © 2008 Elsevier B.V. All rights reserved.
doi:10.1016/j.knosys.2008.03.011

main memory. The algorithm's support count is performed on the cluster table and do not need to scan all the transactions stored in the cluster table.

In Han et al. [14] introduced a novel algorithm, known as the FP-growth method, for mining frequent itemsets. The FP-growth method is a depth-first search algorithm. In the method, a data structure called the FP-tree is used for storing frequency information of the original database in a compressed form. Only two database scans are needed for the algorithm and no candidate generation is required. This makes the FP-growth method much faster than Apriori.

There are many alternatives and extensions to the FP-growth approach, including H-Mine [15] which explores a hyper-structure mining of frequent itemsets; dynamically considering item order, intermediate result representation, and construction strategy, as well as tree traversal strategy by Liu et al. [16]; and an array-based implementation of prefix-tree-structure for efficient pattern growth mining by Grahne and Zhu [17].

Both the Apriori and FP-growth methods mine frequent itemsets from a set of transactions in *horizontal data format* (i.e., {*tid*: *itemset*}), where *tid* is a transaction-id and itemset is the set of items bought in transaction *tid*. Alternatively, mining can also be performed with data presented in *vertical data format* (i.e., {*item*: *tidset*}).

Zaki proposed Eclat algorithm [18] by exploring the vertical data format. The first scan of the database builds the tidset of each single item. Starting with a single item ($k = 1$), the frequent ($k + 1$)-itemsets grown from a previous k-itemset can be generated according to the Apriori property, with a depth-first computation order. The computation is done by intersection of the tidsets of the frequent k-itemsets to compute the tidsets of the corresponding ($k + 1$)-itemsets. This process repeats, until no frequent itemsets or no candidate itemsets can be found. Later, Zaki and Gouda [19] introduced a technique, called *diffset*, for reducing the memory requirement. The *diffset* technique only keeps track of differences in the tid's of candidate itemsets when it is generating frequent itemsets.

Recently, Dong and Han presented BitTableFI algorithm [20]. In the algorithm, data structure BitTable is used horizontally and vertically to compress database for quick candidate itemsets generation and support counting, respectively. As reported in [20], BitTableFI outperforms other two Apriori-like algorithms.

1.3. Our contributions

In this work, we use the BitTable, that has been shown to be a very efficient data structure for mining frequent itemsets [18–20].

The BitTableFI algorithm achieves good performance gained by (possibly significantly) reducing the cost of candidate generation and support counting. However, in situations with a large number of frequent itemsets, long itemsets, or quite low minimum support thresholds, BitTableFI algorithm may suffer from the following two nontrivial costs.

(1) Although making use of efficient bit wise operations, BitTableFI algorithm still use the framework of candidate generation and test, which leads to high computational cost for processing these candidates.
(2) As stated in [6], for the candidate generation and test algorithms, the candidate set generated during an early iteration is generally, in orders of magnitude, larger than the set of frequent itemsets it really contains. Therefore, the initial candidate set generation is the key issue that really counts. However, for BitTableFI algorithm, the length-2 candidates should be generated in the same way as Apriori does. For example, if there are 10^4 frequent 1-itemsets, the BitTableFI algorithm will need to generate more than 10^7 length-2 candidates. The storage of the bit-vectors of these candidates will lead to high spatial complexity, and the support-counting of these candidates will cause high temporal complexity.

To address this problem, in this paper, we present a new algorithm for discovering frequent itemsets. The main contributions of our approach are as follows:

(1) The data structure BitTable is also used horizontally and vertically. To make use of BitTable horizontally, index array and the corresponding computing method are proposed. By computing the subsume index, those itemsets that co-occurrence with representative item can be identified quickly through using breadth-first search at one time. Then, for the resulting itemsets generated by the index array, depth-first search strategy is used to generate all other frequent itemsets. Thus, the hybrid search is implemented, and the search space is reduced greatly.
(2) It is proved that frequent itemsets, including representative item and having the same support as representative item, can be identified directly by connecting the representative item with all the combinations of items in its subsume index. Thus, the cost for processing this kind of itemsets is lowered, and the efficiency is improved.

1.4. Organization of the paper

The remaining of the paper is organized as follows. In Section 2, we briefly revisit the problem definition of frequent itemset mining. In Section 3, we present the definition of Index Array (IA) and the corresponding algorithm for computing index array. In Section 4, we devise algorithm Index-BitTableFI by exploiting the heuristic information provided by IA. A thorough performance study of Index-BitTableFI in comparison with several recently developed efficient algorithms is reported in Section 5. We conclude this study in Section 6.

2. Problem statement

The problem of mining frequent itemsets is formally stated by Definitions 1–3 and Lemma 1.

Let $\boldsymbol{I} = \{i_1, i_2, \ldots, i_M\}$ be a finite set of items and $\boldsymbol{D}$ be a dataset containing N transactions, where each transaction $t \in \boldsymbol{D}$ is a list of distinct items $t = \{i_1, i_2, \ldots i_{|t|}\}$, $i_j \in \boldsymbol{I} (1 \leqslant j \leqslant |\, t\,|)$, and each transaction can be identified by a distinct identifier *tid*.

Definition 1. A set $X \subseteq \boldsymbol{I}$ is called an *itemset*. An itemset with k items is called a k-itemset.

Definition 2. The support of an itemset X, denoted as $sup(X)$, is defined as the number of transactions in which X occurs as a subset.

Definition 3. For a given $\boldsymbol{D}$, let *min_sup* be the threshold minimum support value specified by user. If $sup(X) \geqslant min\_sup$, itemset X is called a frequent itemset.

The task FIM is to generate all frequent itemsets in the database, which have a support greater than *min_sup*.

Lemma 1. *A subset of any frequent itemset is a frequent itemset, a superset of any infrequent itemset is not a frequent itemset.*

Consider an example database shown in Table 1. There are 14 different items, and the database consists of 10 transactions. For convenience we write an itemset $\{A, B, C\}$ as ABC, and a set of

Table 1
The example database

TID	Items	Ordered items
1	*A B C E F O*	*B F A C E*
2	*A C G*	*G A C*
3	*E I*	*E*
4	*A C D E G*	*D G A C E*
5	*A C E G L*	*G A C E*
6	*E J*	*E*
7	*A B C E F P*	*B F A C E*
8	*A C D*	*D A C*
9	*A C E G M*	*G A C E*
10	*A C E G N*	*G A C E*

transaction identifiers {2, 4, 5} as 245. In the example database, suppose $min\_sup = 2$, *GACE*, whose support is 4, is a frequent itemset.

3. Index array and its generation

The BitTableFI algorithm achieves good performance gained by (possibly significantly) reducing the cost of candidate generation and support counting. However, in situations with a large number of frequent itemsets, long itemsets, or quite low minimum support thresholds, it is costly to handle a huge number of candidate sets. As stated in [6], for the candidate generation and test algorithms, the initial candidate set generation, especially for the frequent 2-itemsets, is the key issue that really counts. However, for BitTableFI algorithm, the length-2 candidates should be generated in the same way as Apriori does. The storage of the bit-vectors of these candidates will lead to high spatial complexity, while the support-counting of these candidates will cause high temporal complexity.

To reduce the search space and inherit the advantage of exploiting BitTable horizontally and vertically, *index array* is proposed. The motivation is to identify those itemsets that co-occurrence with the representative frequent item. The resulting itemsets are all frequent. Thus, the redundant operations on candidate generation and frequency-checking can be avoided.

The concept of index array is based on the following function.

$$g(X) = \{t \in \boldsymbol{D} \mid \forall i \in X, i \in t\}$$

Function g associates with X the transactions related to all items $i \in X$. For example, in the example database in Table 1, $g(BF) = g(B) \cap g(F) = 17$.

Definition 4. An *index array* is an array with size m_1, where m_1 is the number of frequent 1-itemset. Each element of the array corresponds to a tuple (*item*, *subsume*), where *item* is an item, $subsume(item) = \{j \in \boldsymbol{I} \mid item \prec j \wedge g(item) \subseteq g(j)\}$. For each element of index array, we call *item* the *representative item*, and *subsume*(*item*) the *subsume index*.

The *subsume index* of *item* is an itemset, whose meaning is if $j \in subsume(item)$, according to some order "$\prec$" (e.g., lexicographic order), j is after *item*, and the tidset (i.e., sets of identifiers of the transactions which contain a given item) of *item* is the subset of tidset of j.

Based on BitTable, the pseudocode for generating index array is shown in Algorithm 1.

Algorithm 1. Computing index array

Input: dataset $\boldsymbol{D}$, *min_sup*
Output: index array
1: Scan database $\boldsymbol{D}$ once. Delete infrequent items;
2: Sort frequent single items in supports ascending order as $a_1, a_2, .., a_m$;
3: **for** each element *index*[*j*] of index array **do**
4: $index[j].item = a_j$;
5: Represent the database $\boldsymbol{D}$ with BitTable;
6: **for** each element *index*[*j*] in index array **do**
7: $index[j].subsume = \emptyset$;
8: $candidate = \bigcap_{t \in g(index[j].item)} t$;
9: **for** each $i > j$ do
10: **if** the value of the *i*-th bit in *candidate* is set **then**
11: $index[j].subsume \leftarrow index[i].item$;
12: **end if**
13: **end for**
14: **end for**
15: **Write Out** the index array;

In Algorithm 1, the database $\boldsymbol{D}$ is first scanned once to determine the frequent single items (Step 1). In Step 2, frequent items are ordered in support ascending order, and the sorted frequent items are assigned to elements of index array as representative items one by one (Steps 3–4). In Step 5, the BitTable representation of database $\boldsymbol{D}$ is built. That is for a transaction T, if 1-frequent itemset i is contained by T, then the bit corresponding to i in T will be set. The index array is calculated by the main loop (Steps 6–15). New candidate of subsume index is formed by intersecting all transactions containing item $index[j].item$ (Steps 7–8). Then the subsume index is obtained according to Definition 4 (Steps 9–13). Note that, since we sort the frequent items in increasing order of their supports, according to Definition 4, we know that the last ordered item has the highest support. Thus, the calculation of its subsume index can be omitted.

Example 1. We use example database in Table 1 to illustrate the basic idea of Algorithm 1.

Suppose the support threshold *min_sup* is 2, after the first scan of database, infrequent items are deleted. Then we sort the list of frequent items in support ascending order (If two items have the same supports, they will be sorted according to lexicographic order). The sorted transactions are shown in Table 1. Then the scanned database is represented by BitTable (shown in Fig. 1).

Then calculate the intersection of transactions that contain certain frequent item one by one. We take frequent item *B* as example, $candidate = \cap_{t \in g(B)} t = 1 \cap 7 = 1010111 \cap 1010111 = 1010111$, where 1 and 7 are tids. There are five 1 in *candidate*, since the first bit corresponds to *B*, the items, corresponds the third, the fifth, the sixth and the last bit, constitute the subsume index of *B*, that is *FACE*. We can iterate this process similarly, and finally the index array is $(B, FACE)$, (D, AC), (F, ACE), (G, AC), (A, C), $(C, \varnothing)$, $(E, \varnothing)$.

4. Index-BitTableFI Algorithm

The pseudocode of Index-BitTableFI is shown in Algorithm 2.

In Algorithm 2, the representative items and their supports are written out at first (Step 2). In Step 3, we determine whether the subsume index of certain representative item is empty. If it is

	B	*D*	*F*	*G*	*A*	*C*	*E*
1	1	0	1	0	1	1	1
2	0	0	0	1	1	1	0
3	0	0	0	0	0	0	1
4	0	1	0	1	1	1	1
5	0	0	0	1	1	1	1
6	0	0	0	0	0	0	1
7	1	0	1	0	1	1	1
8	0	1	0	0	1	1	0
9	0	0	0	1	1	1	1
10	0	0	0	1	1	1	1

Fig. 1. BitTable representation of example database.

empty, then depth-first extension is used when the support of representative item is higher than minimum support threshold (Steps 4–5). Here and in the following Step 11, depth-first extension will not be called, unless the support of representative item is higher than *min_sup*. This is because we have the following Theorem 1. If the subsume index of representative item is not empty, we combine the representative item with every nonempty subset of its subsume index to form frequent itemset with the support being the support of the representative item (Steps 8–9). That is, for certain representative item *index*[*j*].*item* and its subsume index $a_1, a_2, .., a_m$, we combine *index*[*j*].*item* with the $2^m - 1$ nonempty subsets of $a_1, a_2, \ldots, a_m$, the resulting itemsets are all frequent with the same supports as *sup*(*index*[*j*].*item*). This is because we have the following Theorem 2. Similar to Step 4, Step 11 determines whether we should extend the enumerating representative item by depth-first manner. If the condition is satisfied, then the items in *index*[*j*].*subsume* will not be considered for the extension of *index*[*j*].*item* (Step 12). In Step 13, procedure Depth_First is called to extend *index*[*j*].*item*. Then *index*[*j*].*item* is combined with the $2^m - 1$ nonempty subsets of its subsume index $a_1, a_2, \ldots, a_m$, and the resulting frequent itemsets will also be extended in depth-first manner. In procedure Depth_First, we traverse the search space of FIM in pure depth-first order. Note that the support counting method in Algorithm 2 is the same as that used in BitTableFI [20]. That is support of any *k*-itemset is determined by intersecting tid-lists of two of its $(k-1)$-subsets.

Despite having many advantages, the algorithms which use vertical format need more time to perform the intersection operations and also need more memory for storing the tidsets. To avoid these drawbacks, as in dEclat[19], *diffset* technique is also used in Index-BitTableFI. Instead of storing the entire tidsets, *diffset* only keeps track of differences in the tids of a candidate pattern from its generating frequent patterns. Zaki and Gauda [19] reported that the memory required to store the *diffsets* is much smaller than that of the tidsets.

Theorem 1. *Let index*[*j*].*item be a frequent itemset with* $sup(index[j].item) = min\_sup$, *then there exists no item* $index[j].item \prec i$ *and* $i \notin index[j].subsume$, *such that* $index[j].item \cup i$ *is a frequent itemset.*

Proof. We prove the theorem by contradiction. Suppose there exists $index[j].item \prec i$ and $i \notin index[j].subsume$, such that $index[j].item \cup i$ is a frequent itemset. Since function *g* (described in Section 3) is monotonous decreasing, and $index[j].item \subseteq index[j].\ item \cup i$, we have $g(index[j].item \cup i) \subseteq g(index[j].item)$, i.e., $sup(index[j].item \cup i) \leqslant sup(index[j].item)$. Assume $sup(index[j]\ .item \cup i) = sup(index[j].item)$, this means $g(index[j].item) = g(index[j].item \cup i) = g(index[j].item) \cap g(i)$. Thus, $g(index[j].item) \subseteq g(i)$. According to Definition 4, we can easily find that $i \in index[j].subsume$. Thus, $sup(index[j].item \cup i) < sup(index[j].\ item) = min\_sup$. So we can draw the conclusion that there exists no item $index[j].item \prec i$ and $i \notin index[j].subsume$, such that $index[j].item \cup i$ is a frequent itemset. □

Algorithm 2. Index-BitTableFI Algorithm

Input:index array, *min_sup*
Output:frequent itemsets
1: **for** each element *index*[*j*] of index array **do**
2: **Write Out** *index*[*j*].*item* and its support;
3: **if** *index*[*j*].*subsume* $== \emptyset$ **then**
4: **if** $(sup(index[j].item) > min\_sup)$ **then**
5: Depth_First(*index*[*j*].*item*, $t(index[j].item)$);
// *t(index[j].item)* is the set of frequent single items that after *index[j].item*, according to support ascending order
6: **end if**
7: **else**
8: **for** each element $s - item \subseteq index[j].subsume$ **do**
9: **Write Out** $index[j].item \cup s - item$ and its support;
// The support of any $index[j].item \cup s - item$ equals to support of *index[j].item*
10: **end for**
11: **if** $(sup(index[j].item) > min\_sup)$ **then**
12: $tail \leftarrow t(index[j].item) \setminus$ items in *index[j].subsume*;
//delete items included by *index*[*j*].*subsume* from *t(index[j].item)*
13: Depth_First(*index*[*j*].*item*, *tail*);
14: **for** each element $s - item \subseteq index[j].subsume$ **do**
15: Depth_First(*index*[*j*].*item*∪*s-item*, *tail*);
16: **end for**
17: **end if**
18: **end else**
19: **end if**
20: **end for**
Procedure Depth_First (*itemset*, *tail*)
21: **if** $tail == \emptyset$ **then return**;
22: **for** each $i \in tail$ **do**
23: $f - itemset \leftarrow itemset \cup i$;
24: **if** $sup(f - itemset) \geqslant min\_sup$ **then**
25: **Write Out** *f-itemset* and its support;
26: $tail \leftarrow tail \setminus i$;
27: Depth_First (*f-itemset*, *tail*);
28: **end if**
29: **end for**

Theorem 2. *Let item be a representative item,* $subsume(item) = a_1, a_2, .., a_m$, *if item is combined with the* $2^m - 1$ *nonempty subsets of* $a_1, a_2, .., a_m$, *the supports of the resulting itemsets are all* $sup(item)$.

Proof. Let $s - item = b_1, b_2, .., b_k (1 \leqslant k \leqslant m)$ be a nonempty itemset, where any $b_j \in subsume(item) (1 \leqslant j \leqslant k)$. That is $s - item \subseteq 2^{a1,a2,\ldots,am}$, where $2^{a1,a2,\ldots,am}$ is the power set of *subsume*(*item*). According to Definition 4, for $\forall b_j \in s - item$, we have $g(item) \subseteq g(b_j)$. Thus,

$g(item \cup s - item) = g(item) \cap g(s - item) = g(item) \cap g(b_1) \cap g(b_2) \cap \ldots \cap g(b_k) = g(item)$

Therefore, $sup(item \cup s - item) = sup(item)$.

Algorithm Correctness. The main improvement of our algorithm is to optimize the frequent single items and those items co-occurrence with them. For any enumerating item *i*, there are two possible cases when extending it with item $i \prec j$, according to support ascending order:

(1) If $j \in subsume(i)$, Algorithm 2 writes out $i \cup j$ with $sup(i)$. The correctness of this case is confirmed by Theorem 2. Then both *i* and other similar combination results will be extended by simple depth-first order.
(2) If $j \notin subsume(i)$, Algorithm 2 enumerates frequent itemsets in simple depth-first order.

So we can see that Index-BitTableFI enumerates all frequent itemsets completely and correctly.

Example 2. We illustrate Index-BitTableFI algorithm on example database in Table 1.

After the processing of Algorithm 1, the index array is $(B, FACE)$, (D, AC), (F, ACE), (G, AC), (A, C), $(C, \emptyset)$, $(E, \emptyset)$.

B:2 is output at first (the number after ":" indicates the support). Since the subsume index of B is not empty, we combine B with all the nonempty subsets of its subsume index $FACE$, all these resulting itemsets have the same supports as that of B. Thus, the following frequent itemsets will be generated: BF:2, BA:2, BC:2, BE:2, BFA:2, BFC:2, BFE:2, BAC:2, BAE:2, BCE:2, $BFAC$:2, $BFAE$:2, $BFCE$:2, $BACE$:2, $BFACE$:2. Because $sup(B) = min\_sup$, it will not be expanded any more. Note that for generating the above 15 frequent itemsets, the level wise manner used by BitTableFI will store all the candidates with length shorter than 4. Although the frequency checking is efficient using bit operations, the large number of candidates can still lead to high cost. Here we can see that our Index-BitTableFI algorithm avoids the redundant operations on generation and support calculation of them.

Then, for the elements of index array whose representative items are D and F, Index-BitTableFI exploits the similar processes as that of B. Thus, the following frequent itemsets will be generated: D:2, DA:2, DC:2, DAC:2; F:2, FA:2, FC:2, FE:2, FAC:2, FAE:2, FCE:2, $FACE$:2.

Next, for the element of index array whose representative item is G, G:5 is written out at first. Then the following frequent itemsets can be generated by combining G with all the nonempty subsets of its subsume index AC: GA:5, GC:5, GAC:5. According to support ascending order, the set of items after G is $t(G) = ACE$. As $sup(G) > min\_sup$ and both A and C are in $subsume$ (G), items A and C will be deleted in $t(G)$. Then G is extended in a depth first manner, and GE:4 is generated. Since $tail$ is empty. The resulting itemsets GA, GC and GAC will be extended respectively. Thus, the following frequent itemsets will be generated: GAE:4, GCE:4, $GACE$:4.

Then, for the remaining elements of index array whose representative items are A, C and E, Index-BitTableFI iterates the above-mentioned processes similarly. The following frequent itemsets will be generated: A:8, AC:8, AE:6, ACE:6; C:8, CE:6; E:8.

The search space of Algorithms 1 and 2 over example database is shown in Fig. 2.

5. Performance evaluation

We compared the performances of Index-BitTableFI with algorithms Apriori [2], CBAR [13] and BitTableFI [20]. Index-BitTableFI, along with Apriori, CBAR and BitTableFI are implemented in C++ and compiled with Microsoft Visual C++ 6.0. In this sets of experiments, we confirmed the conclusion made at the FIMI 2003 workshop [21], that there are no clear winners with all databases. Indeed, algorithms that were shown to be winners with some databases were not the winners with others. Some algorithms quickly lose their lead once the support level becomes smaller.

5.1. Test environment and datasets

We chose several real and synthetic datasets for testing the performance of Index-BitTableFI. All datasets are taken from the FIMI repository page http://fimi.cs.helsinki.fi. The connect and chess datasets are derived from their respective game steps. Typically, these real datasets are very dense, i.e., they produce many long frequent itemsets even for very high values of support. We also chose a few synthetic datasets, which have been used as benchmarks for testing previous association mining algorithms. These datasets mimic the transactions in a retailing environment. Usually the synthetic datasets are sparse when compared to the real sets. Table 2 shows the characteristics of these datasets. The experiments were conducted on a Windows XP PC equipped with a Pentium 1.5 GHz CPU and 1 GB of RAM memory.

5.2. The runtime

All of the figures use total running time as the performance metric. Because all of the datasets are relatively small, the time to load and prepare the data is negligible and, therefore, the total running time reflects the algorithmic performance only.

Fig. 3 shows the results of comparing Index-BitTableFI with Apriori, CBAR and BitTableFI on sparse data.

On the sparse artificial datasets, BitTableFI demonstrates the best performance of the four algorithms for higher supports. The question of this situation on a synthetic dataset can be answered as follows: For Index-BitTableFI, there are not so many items that co-occurrence together for a sparse dataset when the support threshold is high. Thus, the effect of using index array is not nota-

Table 2
Characteristics of datasets used for experiment evaluations

	# Items	# Records	Avg. length
Chess	75	3196	37
Connect	129	65,557	43
T10I4D100K	870	100,000	11
T40I10D100K	942	100,000	40.5

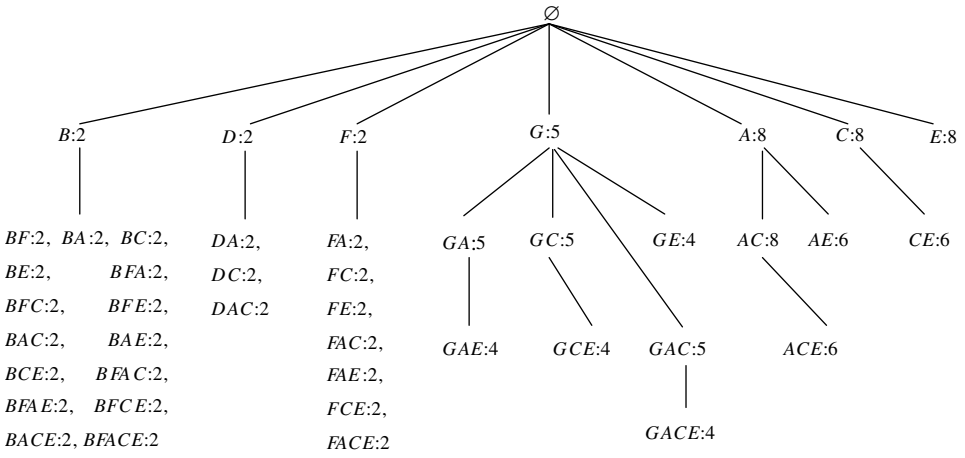

Fig. 2. Search space of Algorithms 1 and 2 over example database.

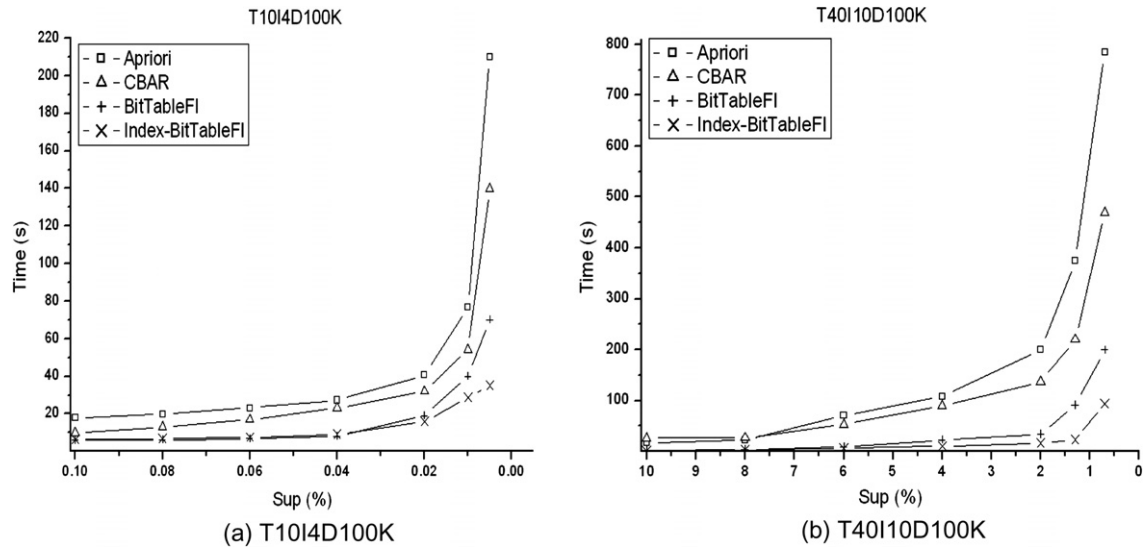

Fig. 3. Execution times (in seconds) required by Index-BitTableFI, Apriori, CBAR and BitTableFI to mine various publicly available sparse datasets as a function of the minimum support threshold.

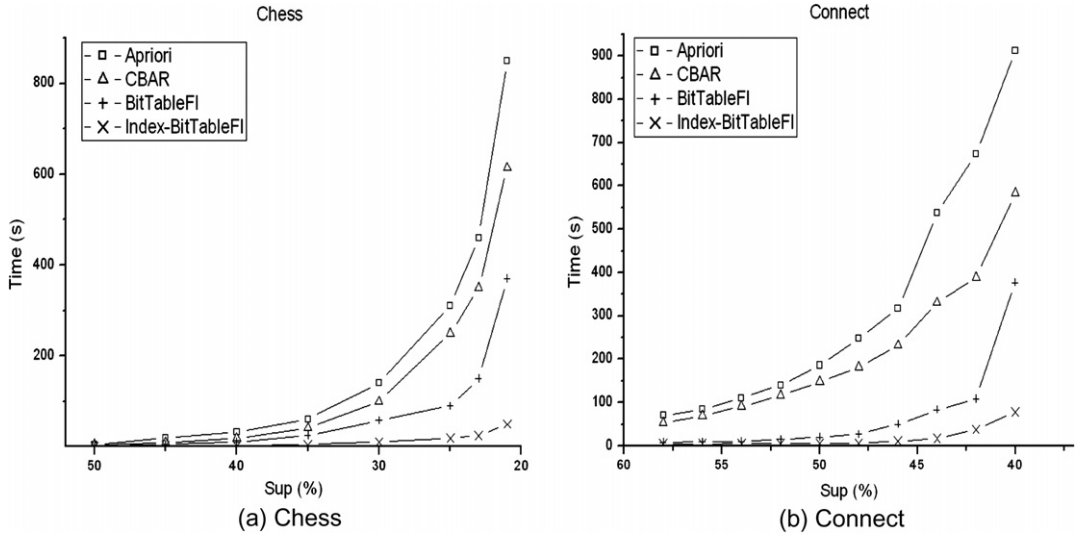

Fig. 4. Execution times (in seconds) required by Index-BitTableFI, Apriori, CBAR and BitTableFI to mine various publicly available dense datasets as a function of the minimum support threshold.

ble. However, note that, as the support drops and the itemsets become longer, Index-BitTableFI passes BitTableFI in performance to become the fastest algorithm. It is clear that Index-BitTableFI performs best when the itemsets are longer.

The dense datasets in Fig. 4 support the idea that Index-BitTableFI runs the fastest on longer itemsets. For most supports on the dense datasets, Index-BitTableFI has the best performance. Generally, Index-BitTableFI runs around two to six times faster than BitTableFI.

6. Conclusions

The BitTableFI algorithm achieves good performance gained by reducing the cost of candidate generation and support counting. However, in situations with a large number of frequent itemsets, long itemsets, or quite low minimum support thresholds, it is costly to handle a huge number of candidate sets. To address this problem, in this paper, a new algorithm, Index-BitTableFI is proposed. Similar to BitTableFI, the data structure BitTable is also used horizontally and vertically to calculate the index array and count supports, respectively. Index array and the corresponding computing method are proposed. By computing the subsume index, those itemsets that co-occurrence with representative item can be identified quickly. Furthermore, it is proved that frequent itemsets, including representative item and having the same support as representative item, can be identified directly by connecting the representative item with all the combinations of items in its subsume index. Thus, the cost for processing this kind of itemsets is lowered, and the efficiency is improved. The experimental results show that our technique works especially well for dense data sets.

The BitTable-based algorithm can not perform well, until there is enough memory to keep the BitTable representation of input dataset. However, real world datasets may be huge. So to design efficient out-core algorithm for mining frequent itemset is our future work. Furthermore, the set of frequent itemsets derived by

most of the current itemset mining methods is too huge for effective usage. There are proposals on reduction of such a huge set, including closed itemsets [22], maximal itemsets [23], approximate itemsets [24], and nonderivable itemset [25], etc. However, it is still not clear what kind of itemsets will give us satisfactory itemsets in both compactness and representative quality for a particular application. So to derive a *compact but high quality* itemsets that are most useful in applications is our another future work.

Acknowledgement

This work is supported by the National Natural Science Foundation of PR China (60675030), and Funding Project for Academic Human Resources Development in Institutions of Higher Learning Under the Jurisdiction of Beijing Municipality.

References

[1] R. Agrawal, T. Imielinski, A. Swami, Mining associations between sets of items in massive databases, in: Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data (SIGMOD'93), Washington, DC, 1993, pp. 207–216.
[2] R. Agrawal, R. Srikant, Fast algorithms for mining association rules in large databases, in: Proceedings of the 20th International Conference on Very Large Data Bases (VLDB'94), Chile, 1994, pp. 487–499.
[3] R. Agrawal, R. Srikant, Mining sequential patterns, in: Proceedings of the Eleventh International Conference on Data Engineering (ICDE'95), Taipei, 1995, pp. 3–14.
[4] H. Mannila, H. Toivonen, A.I. Verkamo, Discovery of frequent episodes in event sequences, Data Mining and Knowledge Discovery 1 (3) (1997) 259–289.
[5] A. Inokuchi, T. Washio, H. Motoda, Complete mining of frequent patterns from graphs: mining graph data, Machine Learning 50 (3) (2003) 321–354.
[6] J.S. Park, M.S. Chen, P.S. Yu, An efficient hash-based algorithm for mining association rules, in: Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data (SIGMOD'95), San Jose, California, 1995, pp. 175–186.
[7] A. Savasere, E. Omiecinski, S.B. Navathe, An efficient algorithm for mining association rules in large databases, in: Proceedings of 21th International Conference on Very Large Data Bases (VLDB'95), Zurich, 1995, pp. 432–444.
[8] H. Toivonen, Sampling large databases for association rules, in: Proceedings of 22th International Conference on Very Large Data Bases (VLDB'96), Bombay, India, 1996, pp. 134–145.
[9] S. Brin, R. Motwani, J.D. Ullman, S. Tsur, Dynamic itemset counting and implication rules for market basket data, in: Proceedings ACM SIGMOD International Conference on Management of Data (SIGMOD'97), Tucson, Arizona, 1997, pp. 255–264.
[10] J. Fong, H.K. Wong, S.M. Huang, Continuous and incremental data mining association rules using frame metadata model, Knowledge Based Systems 16 (2) (2003) 91–100.
[11] S. Sarawagi, S. Thomas, R. Agrawal, Integrating mining with relational database systems: alternatives and implications, in: Proceedings ACM SIGMOD International Conference on Management of Data (SIGMOD'98), Seattle, Washington, 1998, pp. 343–354.
[12] S. Orlando, P. Palmerini, R. Perego, Enhancing the Apriori algorithm for frequent set counting, in: Proceedings of Third International Conference on Data Warehousing and Knowledge Discovery (DaWak'01), Munich, 2001, pp. 71–82.
[13] Y.J. Tsay, J.Y. Chiang, CBAR: an efficient method for mining association rules, Knowledge Based Systems 18 (2–3) (2005) 99–105.
[14] J. Han, J. Pei, Y. Yin, Mining frequent patterns without candidate generation, in: Proceedings of the 2000 ACM-SIGMOD International Conference on Management of Data (SIGMOD'00), Dallas, Texas, 2000, pp. 1–12.
[15] J. Pei, J. Han, H. Lu, S. Nishio, S. Tang, D. Yang, H-Mine: hyper-structure mining of frequent patterns in large databases, in: Proceedings of the 2001 IEEE International Conference on Data Mining (ICDM'01), San Jose, CA, 2001, pp. 441–448.
[16] G. Liu, H. Lu, W. Lou, Y. Xu, J.X. Yu, Efficient mining of frequent patterns using ascending frequency ordered prefix-tree, Data Mining Knowledge Discovery 9 (3) (2004) 249–274.
[17] G. Grahne, J. Zhu, Fast algorithms for frequent itemset mining using FP-Trees, IEEE Transaction on Knowledge and Data Engineering 17 (10) (2005) 1347–1362.
[18] M.J. Zaki, Scalable algorithms for association mining, IEEE Transactions on Knowledge and Data Engineering 12 (3) (2000) 372–390.
[19] M.J. Zaki, K. Gouda, Fast vertical mining using diffsets, in: Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Washington, DC, 2003, pp. 326–335.
[20] J. Dong, M. Han, BitTableFI: an efficient mining frequent itemsets algorithm, Knowledge Based Systems 20 (4) (2007) 329–335.
[21] B. Goethals, M.J. Zaki, Advances in frequent itemset mining implementations introduction to FIMI03, in: Proceedings of the ICDM 2003 Workshop on Frequent Itemset Mining Implementations (FIMI'03), 2003.
[22] U. Yun, Mining lossless closed frequent patterns with weight constraints, Knowledge Based Systems 20 (1) (2007) 86–97.
[23] K. Srikumar, B. Bhasker, Metamorphosis: mining maximal frequent sets in dense domains, International Journal on Artificial Intelligence Tools 14 (3) (2005) 491–505.
[24] J. Liu, S. Paulsen, X. Sun, W. Wang, A.B. Nobel, J. Prins, Mining approximate frequent itemsets in the presence of noise: algorithm and analysis, in: Proceedings of 2006 SIAM International Conference on Data Mining (SDM'06), Bethesda, MD, 2006, pp. 405–416.
[25] T. Calders, B. Goethals, Non-derivable itemset mining, Data Mining and Knowledge Discovery 14 (1) (2007) 171–206.